

JavaScript 핵심 문법 학습 노트

변수부터 비동기 처리까지, 모던 자바스크립트(ES6+) 기본 문법 요약서

1. 변수 선언과 데이터 타입

JavaScript에서 변수를 선언할 때는 스코프와 재할당 가능 여부에 따라 **let** 과 **const** 를 사용합니다.

키워드	스코프 (Scope)	값 재할당	특징
const	블록 스코프	불가능 (상수)	기본적으로 가장 권장되는 선언 방식
let	블록 스코프	가능 (변수)	값이 바뀔 가능성이 있는 변수에 사용
var	함수 스코프	가능	호이스팅 및 중복 선언 문제로 사용 지양

데이터 타입 (Data Types)

- 원시 타입 (Primitive Types): **Number**, **String**, **Boolean**, **null**, **undefined**, **Symbol** (값 자체가 메모리에 저장되며 불변성을 가짐).
- 참조 타입 (Reference Types): **Object**, **Array**, **Function** (메모리 주소를 참조).

2. 연산자와 조건문/반복문

일치 연산자 (Equality)

- **==** (동등 연산자): 암묵적 타입 변환을 허용하여 비교하므로 위험합니다 (예: **1 == '1'** 은 true).
- **===** (일치 연산자): **값과 데이터 타입이 모두 같아야** true를 반환하므로 언제나 이 연산자를 사용해야 합니다.

제어문 체계

```
// 1. 조건문 (if-else, 삼항 연산자)
const score = 85;
const result = score >= 80 ? '합격' : '불합격'; // 삼항 연산자

// 2. 반복문 (for...of, for...in)
const colors = ['red', 'green', 'blue'];
for (const color of colors) {
  console.log(color); // 배열의 요소를 차례대로 순회
}

const user = { name: 'Alice', age: 25 };
for (const key in user) {
  console.log(`${key}: ${user[key]}`); // 객체의 프로퍼티 키를 순회
}
```

3. 함수와 화살표 함수 (Arrow Functions)

ES6에서 도입된 화살표 함수는 함수를 간결하게 표현할 수 있으며, 자신만의 `this` 를 가지지 않고 부모 스코프의 `this` 를 그대로 상속(Lexical this)받는 특징이 있습니다.

```
// 일반 함수 선언식
function multiply(a, b) {
  return a * b;
}

// 화살표 함수 표현식
const multiplyArrow = (a, b) => a * b; // 중괄호와 return 생략 가능
```

4. 자바스크립트 핵심 배열 메서드

모던 자바스크립트에서는 내장 배열 메서드를 활용한 선언적 프로그래밍을 선호합니다. 모든 메서드는 원본 배열을 변형하지 않고 새로운 값을 반환합니다.

- `map()` : 배열의 모든 요소를 돌며 콜백 함수가 반환한 값들로 이루어진 **새로운 배열**을 만듭니다.
- `filter()` : 콜백 함수의 조건이 **true**인 **요소만 걸러내어** 새로운 배열을 만듭니다.
- `reduce()` : 배열의 요소를 하나씩 줄여가며 누적 계산을 수행하여 **하나의 최종 결과값**을 도출합니다.

```
const numbers = [1, 2, 3, 4, 5];

// map: 각 요소에 2를 곱함
const doubled = numbers.map(num => num * 2); // [2, 4, 6, 8, 10]

// filter: 짝수만 추출
const evens = numbers.filter(num => num % 2 === 0); // [2, 4]

// reduce: 배열의 모든 요소 합산
const sum = numbers.reduce((acc, cur) => acc + cur, 0); // 15
```

5. 비동기 처리 (Asynchronous JavaScript)

JavaScript는 싱글 스레드로 동작하므로, 네트워크 요청이나 타이머 같은 오래 걸리는 작업은 비동기로 처리하여 브라우저가 멈추지 않도록 합니다.

비동기 처리의 발전 과정

초기에는 **Callback 함수**를 썼으나 코드가 깊어지는 콜백 지옥(Callback Hell) 문제가 발생했습니다. 이를 해결하기 위해 **Promise**가 도입되었으며, 현재는 동기식 코드처럼 직관적으로 읽히는 **async / await** 문법이 표준으로 널리 사용됩니다.

Promise와 async/await 실전 패턴

```
// 1. Promise 기반의 가상 데이터 페칭 함수
const fetchUserData = (userId) => {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (userId) {
        resolve({ id: userId, name: 'Kim' });
      } else {
        reject(new Error('유효하지 않은 ID입니다.'));
      }
    }, 1000);
  });
};

// 2. async/await과 예외 처리(try-catch) 패턴
async function loadUser() {
  try {
    console.log('데이터 로딩 시작...');
    const user = await fetchUserData(101); // 비동기 작업 완료를 기다림
    console.log('성공:', user.name);
  } catch (error) {
    console.error('에러 발생:', error.message);
  } finally {
    console.log('작업 종료');
  }
}

loadUser();
```